Universidad de los Andes

Departamento de Ingeniería Industrial · Modelado de Sistemas bajo Incertidumbre

Simulación de Eventos Discretos: Mecánica, Trazas y Worldviews

Alejandra Tabares Pozos

Índice

1. ¿Qué es la simulación de eventos discretos? 3

1.1. ¿Cuándo usar SED? . . . 3

2. Formalismo y conexión con procesos estocásticos 3

3. Elementos constitutivos de un modelo SED 4

3.1. Mecanismo de avance del tiempo . . . 4

4. Métricas en sistemas de colas 4

5. Ejemplo manual: cola con un servidor 5

5.1. Bitácora paso a paso . . . 5

5.2. Cálculo de métricas . . . 5

6. Event-Scheduling granular: cola con dos servidores 6

6.1. Definición del estado, la FEL y las rutinas . . . 6

6.2. Datos de entrada . . . 6

6.3. Inicialización (Paso 0) . . . 7

6.4. Ejecución paso a paso . . . 7

6.5. Tabla resumen de áreas . . . 10

6.6. Métricas finales del ejemplo con 2 servidores . . . 10

7. Process-Interaction: el mismo ejemplo, otra mirada 10

7.1. Traza del ejemplo bajo Process-Interaction . . . 11

8. Conexión con el análisis de estado estable 11

1. ¿Qué es la simulación de eventos discretos?

Un modelo de **simulación de eventos discretos** (SED) describe un sistema dinámico cuyo *estado* cambia únicamente en un conjunto discreto de instantes de tiempo llamados **eventos**. Entre eventos, el estado permanece constante. A diferencia de una simulación de tiempo continuo (donde la dinámica se describe mediante ecuaciones diferenciales) o de una simulación en tiempo discreto con paso fijo Δt, en SED la granularidad temporal está dictada por la *lógica del sistema*: el reloj salta directamente de un evento al siguiente sin necesidad de iterar en intervalos artificiales.

La idea central es generar **trayectorias muestrales** del proceso de estado $X(t)$ y, a partir de ellas, aproximar medidas de desempeño que típicamente se expresan como esperanzas: $\theta = \mathbb{E}[g(X)]$, por ejemplo, la espera promedio en cola, la utilización de un servidor o el nivel de inventario. Los cambios de estado son instantáneos — una llegada, una salida, una falla, una reparación — y es precisamente la secuencia de estos cambios lo que le da estructura al modelo.

1.1. ¿Cuándo usar SED?

SED es apropiada cuando existe *interacción compleja* entre componentes (colas, recursos compartidos, prioridades, bloqueos), cuando hay *aleatoriedad estructural* (llegadas, servicios, fallas) que hace intratable el análisis exacto, cuando interesan métricas no lineales o condicionales (percentiles, niveles de servicio, costos con penalizaciones) o cuando se requieren *políticas operacionales* difíciles de analizar de forma cerrada. En cambio, no se justifica usar SED si existe un modelo analítico confiable que responda la pregunta (como una cola $M/M/1$ clásica), si el problema es determinístico o estático, o si no hay datos mínimos para parametrizar la incertidumbre.

2. Formalismo y conexión con procesos estocásticos

Desde el punto de vista matemático, SED construye trayectorias de un proceso estocástico. Considere un estado $X(t) \in \mathcal{X}$ que resume toda la información necesaria para predecir el futuro del sistema. Existen tiempos de evento $0 = T_0 < T_1 < T_2 < \cdots$ tales que $X(t) = X(T_k)$ para $t \in [T_k, T_{k+1})$, y el salto en cada evento se obtiene mediante una regla de la forma

$$X_{k+1} = \Phi(X_k,\, e_{k+1},\, U_{k+1}),$$

donde $X_k := X(T_k)$, e_{k+1} denota el tipo de evento y U_{k+1} representa las variables aleatorias generadas (por ejemplo, un tiempo de servicio). Así, SED es una manera algorítmica de construir una trayectoria muestral de $X(t)$.

Los tiempos de evento $\{T_k\}$ inducen un **proceso puntual** y su proceso contador $N(t) = \text{máx}\{k : T_k \leq t\}$. La dinámica puede estudiarse en dos escalas: en *tiempo real* t (proceso continuo con saltos) o en *épocas de evento* k (tiempo discreto). En esta segunda escala, la sucesión $\{X_k\}$ frecuentemente constituye una **cadena de Markov en tiempo discreto**, lo cual conecta SED con la teoría de colas, cadenas de Markov y procesos de renovación.

3. Elementos constitutivos de un modelo SED

Todo modelo de SED se construye a partir de un conjunto de bloques estándar. Las **entidades** son los objetos que fluyen por el sistema (clientes, piezas, llamadas). Cada entidad posee **atributos** como su clase, prioridad o tiempo de llegada. Los **recursos** son elementos de capacidad limitada que prestan servicio (servidores, máquinas, operadores). Las **colas** son buffers de espera asociados a los recursos. El **estado** es el conjunto mínimo de variables que describe el sistema en cada instante: ocupación de servidores, longitud de cola, eventos programados. Los **eventos** son ocurrencias instantáneas que cambian el estado (una llegada, un fin de servicio). Finalmente, las **actividades** tienen una duración conocida o aleatoria que se programa explícitamente, mientras que las **demoras** son esperas condicionadas por el estado del sistema.

El corazón computacional de cualquier simulador SED es el **calendario de eventos futuros** (FEL, por *Future Event List*): una estructura de datos — típicamente una cola de prioridad ordenada por tiempo — que mantiene los próximos eventos con sus tiempos de ocurrencia.

3.1. Mecanismo de avance del tiempo

El esquema dominante en SED es el **avance al siguiente evento** (*next-event time advance*). El ciclo es: (1) seleccionar el evento más próximo de la FEL, $t^* = \text{mín}_e\, t_e$; (2) avanzar el reloj, $t \leftarrow t^*$; (3) actualizar las estadísticas acumuladas (áreas bajo trayectorias); (4) ejecutar la rutina del evento correspondiente, que a su vez actualiza el estado y puede programar nuevos eventos en la FEL. Este ciclo se repite hasta que se satisface un criterio de parada (por ejemplo, un número mínimo de clientes atendidos o un horizonte temporal fijado).

4. Métricas en sistemas de colas

En una cola, los procesos naturales son $N(t)$, el número de entidades en el sistema en el instante t, y $Q(t)$, el número en cola (esperando). Las métricas se clasifican en dos grandes familias.

Las **métricas promediadas en el tiempo** se calculan como integrales:

$$\bar{N}_T = \frac{1}{T}\int_0^T N(t)\,dt, \qquad \bar{Q}_T = \frac{1}{T}\int_0^T Q(t)\,dt, \qquad \hat{\rho} = \frac{1}{T}\int_0^T \mathbf{1}\{S(t) = \text{BUSY}\}\,dt.$$

Aquí $\bar{N}_T$ es el número promedio de entidades en el sistema, $\bar{Q}_T$ es la longitud promedio de la cola y $\hat{\rho}$ es la utilización del servidor. En la práctica, dado que el estado es constante entre eventos, estas integrales se reducen a sumas de rectángulos: si $\Delta_k = t_{k+1} - t_k$ es el tiempo entre el evento k y el $k+1$, entonces $\bar{Q}_T = \frac{1}{T}\sum_k Q^{(k)} \cdot \Delta_k$ y análogamente para las demás.

Las **métricas por entidad** se calculan como promedios sobre las observaciones individuales. Para el cliente i con tiempo de llegada A_i, inicio de servicio B_i y salida D_i: la espera en cola es $W_i = B_i - A_i$ y el tiempo en sistema es $T_i = D_i - A_i$. Los estimadores son $\bar{W}_n = \frac{1}{n}\sum_{i=1}^n W_i$ y $\bar{T}_n = \frac{1}{n}\sum_{i=1}^n T_i$.

La diferencia conceptual es importante: los promedios en el tiempo integran *áreas bajo trayectorias*,

mientras que los promedios por cliente agregan *muestras por entidad*.

5. Ejemplo manual: cola con un servidor

Para comprender la mecánica de SED desde dentro, consideremos una cola FCFS con un solo servidor. Definimos el estado como la dupla (S, N_Q), donde $S \in \{\text{IDLE}, \text{BUSY}\}$ es el estado del servidor y N_Q es el número de clientes en cola. La FEL contiene dos tiempos: NA (próxima llegada) y ND (próxima salida). Si no hay salida programada, ND $= \infty$.

Usamos 6 clientes con tiempos entre llegadas X_i y tiempos de servicio S_i dados:

Cliente i	1	2	3	4	5	6
X_i (entre llegadas)	0.0	1.2	0.7	1.3	0.4	1.0
S_i (servicio)	1.1	0.9	2.0	1.2	0.8	1.5

La rutina **Arrival** opera así: al llegar un cliente, se programa la siguiente llegada en la FEL; si el servidor está libre, el cliente entra a servicio de inmediato y se programa su salida; si está ocupado, el cliente se encola. La rutina **Departure** opera al revés: si la cola está vacía, el servidor pasa a IDLE; si no, se extrae al siguiente cliente (FCFS), se calcula su demora $W_i = t - A_i$, y se programa su fin de servicio.

5.1. Bitácora paso a paso

Ejecutando el ciclo completo se obtiene la siguiente traza:

t	Evento	S	N_Q	NA	ND
0.0	Llegada C1	BUSY	0	1.2	1.1
1.1	Salida C1	IDLE	0	1.2	∞
1.2	Llegada C2	BUSY	0	1.9	2.1
1.9	Llegada C3	BUSY	1	3.2	2.1
2.1	Salida C2	BUSY	0	3.2	4.1
3.2	Llegada C4	BUSY	1	3.6	4.1
3.6	Llegada C5	BUSY	2	4.6	4.1
4.1	Salida C3	BUSY	1	4.6	5.3
4.6	Llegada C6	BUSY	2	∞	5.3
5.3	Salida C4	BUSY	1	∞	6.1
6.1	Salida C5	BUSY	0	∞	7.6
7.6	Salida C6	IDLE	0	∞	∞

5.2. Cálculo de métricas

A partir de la bitácora se reconstruyen los tiempos por cliente:

i	A_i	B_i	D_i	W_i	T_i
1	0.0	0.0	1.1	0.0	1.1
2	1.2	1.2	2.1	0.0	0.9
3	1.9	2.1	4.1	0.2	2.2
4	3.2	4.1	5.3	0.9	2.1
5	3.6	5.3	6.1	1.7	2.5
6	4.6	6.1	7.6	1.5	3.0

La espera promedio en cola es $\bar{W} = 4{,}3/6 \approx 0{,}717$ y el tiempo promedio en sistema es $\bar{T} = 11{,}8/6 \approx 1{,}967$. Para las métricas en el tiempo, el cálculo por áreas entre eventos arroja: área bajo $N_Q(t) = 4{,}3$ y área bajo el indicador de servidor ocupado $= 7{,}5$, con horizonte $T = 7{,}6$. Así, $\bar{Q}_T = 4{,}3/7{,}6 \approx 0{,}566$ y $\hat{\rho} = 7{,}5/7{,}6 \approx 0{,}987$.

6. Event-Scheduling granular: cola con dos servidores

Esta sección presenta, paso a paso y con todo detalle, la ejecución del worldview **Event-Scheduling** para una cola FCFS con dos servidores. Siga cada paso con lápiz y papel: dominar esta mecánica es la base para entender cualquier simulador.

6.1. Definición del estado, la FEL y las rutinas

Estado mínimo: el reloj t, los estados de cada servidor $S_1, S_2 \in \{\text{IDLE}, \text{BUSY}\}$, el número en cola N_Q y la cola FCFS que guarda los tiempos de llegada de quienes esperan.

FEL: ahora tiene tres entradas: NA (próxima llegada), ND1 (próxima salida del servidor 1) y ND2 (próxima salida del servidor 2). Si un evento no está programado, su tiempo es ∞.

Rutina Timing: selecciona el mínimo entre NA, ND1 y ND2, avanza el reloj t y determina el tipo de evento.

Rutina ARRIVAL en tiempo t: (1) Programar la próxima llegada: NA $\leftarrow t + X_{i+1}$ (si existe). (2) Si $S_1 = \text{IDLE}$: asignar a servidor 1, $S_1 \leftarrow \text{BUSY}$, programar ND1 $\leftarrow t + S_i$. (3) En caso contrario, si $S_2 = \text{IDLE}$: asignar a servidor 2, $S_2 \leftarrow \text{BUSY}$, programar ND2 $\leftarrow t + S_i$. (4) Si ambos están BUSY: encolar, $N_Q \leftarrow N_Q + 1$, guardar t en la cola FCFS.

Rutina DEPARTURE del servidor j en tiempo t: (1) Si $N_Q = 0$: $S_j \leftarrow \text{IDLE}$, ND$j \leftarrow \infty$. (2) Si $N_Q > 0$: extraer al primero de la cola (FCFS), calcular delay $= t -$ dequeue(), $N_Q \leftarrow N_Q - 1$, asignar al servidor j y programar ND$j \leftarrow t + S_{\text{siguiente}}$.

Rutina Update-Time-Avg-Stats (se ejecuta *antes* de cambiar el estado, en cada evento): sea $\Delta = t - t_{\text{prev}}$, entonces: $\mathcal{A}_Q \mathrel{+}= N_Q \cdot \Delta$ (área bajo la cola), $\mathcal{A}_B \mathrel{+}= B(t_{\text{prev}}) \cdot \Delta$ donde $B = \mathbf{1}\{S_1{=}\text{BUSY}\} + \mathbf{1}\{S_2{=}\text{BUSY}\} \in \{0, 1, 2\}$, y $t_{\text{prev}} \leftarrow t$.

6.2. Datos de entrada

Usamos 7 clientes con tiempos entre llegadas y de servicio fijos:

Cliente i	1	2	3	4	5	6	7
X_i (entre llegadas)	0.0	0.6	0.9	0.4	1.1	0.5	0.7
S_i (servicio)	1.4	1.0	2.2	0.8	1.6	1.3	0.9

6.3. Inicialización (Paso 0)

$t \leftarrow 0$, $S_1 = S_2 = \text{IDLE}$, $N_Q = 0$, cola vacía. Se programa la primera llegada: NA $\leftarrow 0.0$; ND1 = ND2 = ∞. Acumuladores: $\mathcal{A}_Q = \mathcal{A}_B = 0$, $t_{\text{prev}} = 0$.

6.4. Ejecución paso a paso

Paso 1 — Llegada C1 ($t : 0 \to 0.0$)

Timing: $\text{mín}(0.0, \infty, \infty) = 0.0$ (ARRIVAL). Reloj: $t \leftarrow 0.0$.

Update-Stats: $\Delta = 0.0 - 0 = 0$. Sin cambios.

ARRIVAL: S1 libre $\Rightarrow$ C1 entra a S1. $S_1 \leftarrow$ BUSY. Programar ND1 $\leftarrow 0.0 + 1.4 = 1.4$. Programar próxima llegada: NA $\leftarrow 0.0 + 0.6 = 0.6$.

Estado resultante: $(S_1, S_2) = (\text{B}, \text{I})$, $N_Q = 0$. `FEL`: {NA=0.6, ND1=1.4, ND2=∞}.

Paso 2 — Llegada C2 ($t : 0.0 \to 0.6$)

Timing: $\text{mín}(0.6, 1.4, \infty) = 0.6$ (ARRIVAL). Reloj: $t \leftarrow 0.6$.

Update-Stats: $\Delta = 0.6$. $\mathcal{A}_B \mathrel{+}= 1 \times 0.6 = 0.6$. $\mathcal{A}_Q \mathrel{+}= 0$.

ARRIVAL: S1 ocupado, S2 libre $\Rightarrow$ C2 entra a S2. $S_2 \leftarrow$ BUSY. Programar ND2 $\leftarrow 0.6 + 1.0 = 1.6$. Programar NA $\leftarrow 0.6 + 0.9 = 1.5$.

Estado: (B, B), $N_Q = 0$. `FEL`: {NA=1.5, ND1=1.4, ND2=1.6}.

Ambos servidores ocupados. El próximo cliente que llegue deberá esperar.

Paso 3 — Salida S1, C1 ($t : 0.6 \to 1.4$)

Timing: $\text{mín}(1.5, 1.4, 1.6) = 1.4$ (DEPARTURE S1). Reloj: $t \leftarrow 1.4$.

Update-Stats: $\Delta = 0.8$. $\mathcal{A}_B \mathrel{+}= 2 \times 0.8 = 1.6$. $\mathcal{A}_Q \mathrel{+}= 0$.

DEPARTURE S1: C1 sale. $W_1 = 1.4 - 0.0 = 1.4$. Cola vacía ($N_Q = 0$) $\Rightarrow S_1 \leftarrow$ IDLE, ND1 $\leftarrow \infty$.

Estado: (I, B), $N_Q = 0$. `FEL`: {NA=1.5, ND1=∞, ND2=1.6}.

Paso 4 — Llegada C3 ($t : 1.4 \to 1.5$)

Timing: $\text{mín}(1.5, \infty, 1.6) = 1.5$ (ARRIVAL). Reloj: $t \leftarrow 1.5$.

Update-Stats: $\Delta = 0.1$. $\mathcal{A}_B \mathrel{+}= 1 \times 0.1 = 0.1$. $\mathcal{A}_Q \mathrel{+}= 0$.

ARRIVAL: S1 libre $\Rightarrow$ C3 entra a S1. $S_1 \leftarrow$ BUSY. ND1 $\leftarrow 1.5 + 2.2 = 3.7$. NA $\leftarrow 1.5 + 0.4 = 1.9$.

Estado: (B, B), $N_Q = 0$. `FEL`: {NA=1.9, ND1=3.7, ND2=1.6}.

Paso 5 — Salida S2, C2 ($t : 1.5 \to 1.6$)

Timing: $\text{mín}(1.9, 3.7, 1.6) = 1.6$ (DEPARTURE S2). Reloj: $t \leftarrow 1.6$.

Update-Stats: $\Delta = 0.1$. $\mathcal{A}_B \mathrel{+}= 2 \times 0.1 = 0.2$.

DEPARTURE S2: C2 sale. $W_2 = 1.6 - 0.6 = 1.0$. $N_Q = 0 \Rightarrow S_2 \leftarrow$ IDLE, ND2 $\leftarrow \infty$.

Estado: (B, I), $N_Q = 0$. `FEL`: {NA=1.9, ND1=3.7, ND2=∞}.

Paso 6 — Llegada C4 ($t : 1.6 \to 1.9$)

Timing: $\text{mín}(1.9, 3.7, \infty) = 1.9$ (ARRIVAL). Reloj: $t \leftarrow 1.9$.

Update-Stats: $\Delta = 0.3$. $\mathcal{A}_B \mathrel{+}= 1 \times 0.3 = 0.3$.

ARRIVAL: S2 libre $\Rightarrow$ C4 entra a S2. ND2 $\leftarrow 1.9 + 0.8 = 2.7$. NA $\leftarrow 1.9 + 1.1 = 3.0$.

Estado: (B, B), $N_Q = 0$. `FEL`: {NA=3.0, ND1=3.7, ND2=2.7}.

Paso 7 — Salida S2, C4 ($t : 1.9 \to 2.7$)

Timing: $\text{mín}(3.0, 3.7, 2.7) = 2.7$ (DEPARTURE S2). Reloj: $t \leftarrow 2.7$.

Update-Stats: $\Delta = 0.8$. $\mathcal{A}_B \mathrel{+}= 2 \times 0.8 = 1.6$.

DEPARTURE S2: C4 sale. $N_Q = 0 \Rightarrow S_2 \leftarrow$ IDLE, ND2 $\leftarrow \infty$.

Estado: (B, I), $N_Q = 0$. `FEL`: {NA=3.0, ND1=3.7, ND2=∞}.

Hasta aquí ningún cliente ha esperado en cola: siempre había un servidor libre.

Paso 8 — Llegada C5 ($t : 2.7 \to 3.0$)

Timing: $\text{mín}(3.0, 3.7, \infty) = 3.0$ (ARRIVAL). Reloj: $t \leftarrow 3.0$.

Update-Stats: $\Delta = 0.3$. $\mathcal{A}_B \mathrel{+}= 1 \times 0.3 = 0.3$.

ARRIVAL: S2 libre $\Rightarrow$ C5 entra a S2. ND2 $\leftarrow 3.0 + 1.6 = 4.6$. NA $\leftarrow 3.0 + 0.5 = 3.5$.

Estado: (B, B), $N_Q = 0$. `FEL`: {NA=3.5, ND1=3.7, ND2=4.6}.

Paso 9 — Llegada C6 ($t : 3.0 \to 3.5$) — ¡primera cola!

Timing: $\text{mín}(3.5, 3.7, 4.6) = 3.5$ (ARRIVAL). Reloj: $t \leftarrow 3.5$.

Update-Stats: $\Delta = 0.5$. $\mathcal{A}_B \mathrel{+}= 2 \times 0.5 = 1.0$.

ARRIVAL: Ambos servidores BUSY $\Rightarrow$ C6 entra a la cola. $N_Q \leftarrow 1$. Se guarda $A_6 = 3.5$ en la cola FCFS. NA $\leftarrow 3.5 + 0.7 = 4.2$.

Estado: (B, B), $N_Q = 1$, Cola= [3.5]. `FEL`: {NA=4.2, ND1=3.7, ND2=4.6}.

Paso 10 — Salida S1, C3 ($t : 3.5 \to 3.7$) — C6 sale de la cola

Timing: $\text{mín}(4.2, 3.7, 4.6) = 3.7$ (DEPARTURE S1). Reloj: $t \leftarrow 3.7$.

Update-Stats: $\Delta = 0.2$. $\mathcal{A}_B \mathrel{+}= 2 \times 0.2 = 0.4$. $\mathcal{A}_Q \mathrel{+}= 1 \times 0.2 = 0.2$.

DEPARTURE S1: C3 sale. $N_Q = 1 > 0 \Rightarrow$ extraer C6 de la cola. $\text{delay}_6 = 3.7 - 3.5 = 0.2$. $N_Q \leftarrow 0$. C6 entra a S1. ND1 $\leftarrow 3.7 + 1.3 = 5.0$.

Estado: (B, B), $N_Q = 0$. `FEL`: {NA=4.2, ND1=5.0, ND2=4.6}.

Paso 11 — Llegada C7 ($t : 3.7 \to 4.2$) — segunda vez en cola

Timing: $\text{mín}(4.2, 5.0, 4.6) = 4.2$ (ARRIVAL). Reloj: $t \leftarrow 4.2$.

Update-Stats: $\Delta = 0.5$. $\mathcal{A}_B \mathrel{+}= 2 \times 0.5 = 1.0$. $\mathcal{A}_Q \mathrel{+}= 0$.

ARRIVAL: Ambos BUSY $\Rightarrow$ C7 entra a la cola. $N_Q \leftarrow 1$. No hay más llegadas: NA $\leftarrow \infty$.

Estado: (B, B), $N_Q = 1$, Cola= [4.2]. `FEL`: {NA=∞, ND1=5.0, ND2=4.6}.

Paso 12 — Salida S2, C5 ($t : 4.2 \to 4.6$) — C7 sale de la cola

Timing: $\text{mín}(\infty, 5.0, 4.6) = 4.6$ (DEPARTURE S2). Reloj: $t \leftarrow 4.6$.

Update-Stats: $\Delta = 0.4$. $\mathcal{A}_B \mathrel{+}= 2 \times 0.4 = 0.8$. $\mathcal{A}_Q \mathrel{+}= 1 \times 0.4 = 0.4$.

DEPARTURE S2: C5 sale. $N_Q = 1 > 0 \Rightarrow$ extraer C7. $\text{delay}_7 = 4.6 - 4.2 = 0.4$. $N_Q \leftarrow 0$. C7 entra a S2. ND2 $\leftarrow 4.6 + 0.9 = 5.5$.

Estado: (B, B), $N_Q = 0$. `FEL`: {NA=∞, ND1=5.0, ND2=5.5}.

Paso 13 — Salida S1, C6 ($t : 4.6 \to 5.0$)

Timing: $\text{mín}(\infty, 5.0, 5.5) = 5.0$ (DEPARTURE S1). Reloj: $t \leftarrow 5.0$.

Update-Stats: $\Delta = 0.4$. $\mathcal{A}_B \mathrel{+}= 2 \times 0.4 = 0.8$.

DEPARTURE S1: C6 sale. $N_Q = 0 \Rightarrow S_1 \leftarrow$ IDLE, ND1 $\leftarrow \infty$.

Estado: (I, B), $N_Q = 0$. `FEL`: {NA=∞, ND1=∞, ND2=5.5}.

Paso 14 — Salida S2, C7 ($t : 5.0 \to 5.5$) — FIN

Timing: $\text{mín}(\infty, \infty, 5.5) = 5.5$ (DEPARTURE S2). Reloj: $t \leftarrow 5.5$.

Update-Stats: $\Delta = 0.5$. $\mathcal{A}_B \mathrel{+}= 1 \times 0.5 = 0.5$.

DEPARTURE S2: C7 sale. $N_Q = 0 \Rightarrow S_2 \leftarrow$ IDLE, ND2 $\leftarrow \infty$.

Estado: (I, I), $N_Q = 0$. `FEL`: {NA=∞, ND1=∞, ND2=∞}. **Parada.**

6.5. Tabla resumen de áreas

Intervalo	Δ_k	N_Q	B	$N_Q \cdot \Delta$	$B \cdot \Delta$
$[0.0, 0.6)$	0.6	0	1	0.0	0.6
$[0.6, 1.4)$	0.8	0	2	0.0	1.6
$[1.4, 1.5)$	0.1	0	1	0.0	0.1
$[1.5, 1.6)$	0.1	0	2	0.0	0.2
$[1.6, 1.9)$	0.3	0	1	0.0	0.3
$[1.9, 2.7)$	0.8	0	2	0.0	1.6
$[2.7, 3.0)$	0.3	0	1	0.0	0.3
$[3.0, 3.5)$	0.5	0	2	0.0	1.0
$[3.5, 3.7)$	0.2	1	2	0.2	0.4
$[3.7, 4.2)$	0.5	0	2	0.0	1.0
$[4.2, 4.6)$	0.4	1	2	0.4	0.8
$[4.6, 5.0)$	0.4	0	2	0.0	0.8
$[5.0, 5.5)$	0.5	0	1	0.0	0.5
Total	$T = 5.5$			**0.6**	**9.2**

Las filas sombreadas corresponden a los intervalos donde hubo cola ($N_Q > 0$).

6.6. Métricas finales del ejemplo con 2 servidores

$$\bar{Q}_T = \frac{0.6}{5.5} \approx 0.109, \qquad \bar{B} = \frac{9.2}{5.5} \approx 1.673, \qquad \hat{\rho} = \frac{\bar{B}}{2} = \frac{9.2}{2 \times 5.5} \approx 0.836.$$

Solo los clientes 6 y 7 esperaron en cola (0.2 y 0.4 respectivamente), para una espera promedio $\bar{W}_q = 0.6/7 \approx 0.086$.

7. Process-Interaction: el mismo ejemplo, otra mirada

En el worldview **Process-Interaction**, no escribimos rutinas de evento: escribimos el *flujo de vida de una entidad*. El motor de simulación traduce las esperas en eventos internamente. Para nuestra cola con 2 servidores, el proceso de cada cliente se describe así (en pseudocódigo tipo SimPy):

```
proceso Cliente(i):
llegar al sistema en tiempo A_i
solicitar(recurso=Servidor)   // si hay servidor libre, continúa; si no, espera en cola
registrar inicio_servicio = clock
delay_i = inicio_servicio − A_i
esperar(S_i)   // ocupa el servidor durante S_i unidades de tiempo
liberar(recurso=Servidor)
registrar salida = clock
```

El **generador de llegadas** es un proceso separado que crea entidades:

```
proceso Generador:
para i = 1, 2, 3, ...:
esperar(X_i)  // tiempo entre llegadas
crear Cliente(i)
```

El `recurso Servidor` tiene capacidad $c = 2$. Cuando un cliente ejecuta `solicitar(Servidor)`, el motor verifica si hay capacidad disponible. Si la hay, el cliente la toma y continúa su proceso sin interrupción. Si no, el cliente queda suspendido en una cola interna del recurso hasta que otro cliente ejecute `liberar(Servidor)`.

7.1. Traza del ejemplo bajo Process-Interaction

Veamos cómo el motor ejecuta los mismos 7 clientes:

t	Acción	Recurso Servidor	Cola
0.0	C1: solicitar → obtiene (cap. 1/2 usada)	[C1]	vacía
0.6	C2: solicitar → obtiene (cap. 2/2 usada)	[C1, C2]	vacía
1.4	C1: liberar (cap. 1/2 usada)	[C2]	vacía
1.5	C3: solicitar → obtiene (cap. 2/2)	[C2, C3]	vacía
1.6	C2: liberar (cap. 1/2)	[C3]	vacía
1.9	C4: solicitar → obtiene (cap. 2/2)	[C3, C4]	vacía
2.7	C4: liberar (cap. 1/2)	[C3]	vacía
3.0	C5: solicitar → obtiene (cap. 2/2)	[C3, C5]	vacía
3.5	C6: solicitar → espera (cap. llena)	[C3, C5]	[C6]
3.7	C3: liberar → C6 obtiene	[C6, C5]	vacía
4.2	C7: solicitar → espera	[C6, C5]	[C7]
4.6	C5: liberar → C7 obtiene	[C6, C7]	vacía
5.0	C6: liberar	[C7]	vacía
5.5	C7: liberar	[ ]	vacía

La trayectoria es idéntica a la obtenida con Event-Scheduling: los mismos clientes esperan, las mismas demoras se producen, las mismas métricas resultan. La diferencia está en *cómo se expresa la lógica*: en Event-Scheduling el modelador controla la FEL y escribe rutinas de evento; en Process-Interaction el modelador escribe la "biografía" de la entidad y el motor se encarga del resto.

8. Conexión con el análisis de estado estable

Las trazas de esta lectura muestran el comportamiento de pocos clientes en un sistema que parte vacío. En la práctica, interesa el comportamiento del sistema *a largo plazo*: la distribución de equilibrio, el efecto del período transitorio y la Ley de Little. Estos temas se estudian en la lectura complementaria **Sistemas de Colas con SED: Ley de Little, Período Transitorio y**

Validación.

Referencias

[1] Banks, J., Carson, J. S., Nelson, B. L., Nicol, D. M. (2014). *Discrete-Event System Simulation*, 5th ed. Pearson. Cap. 3–4.

[2] Law, A. M. (2015). *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill. Cap. 3.

[3] Kelton, W. D., Sadowski, R. P., & Zupick, N. B. (2015). *Simulation with Arena*, 6th ed. McGraw-Hill. Cap. 2.

[4] Fishman, G. S. (2001). *Discrete-Event Simulation: Modeling, Programming, and Analysis*. Springer. Cap. 2.

[5] Ross, S. M. (2014). *Introduction to Probability Models*, 11th ed. Academic Press. Cap. 4.